

SÉRIE : BASES DE DONNÉES

CHAPITRE 9 : ÉTUDE DE CAS

Analyse, Conception (MCD/MLD),
Implémentation SQL, Requêtes Avancées
et Interfaçage Applicatif



Contents

1	Fiche 1 : Le Cours Complet — Conception & Implémentation	2
1.1	1. Présentation de l'Étude de Cas : Gestion de Médiathèque Scolaire	2
1.2	2. Étape 1 : Le Dictionnaire de Données (DD)	2
1.3	3. Étape 2 : Le Modèle Conceptuel des Données (MCD)	3
1.4	4. Étape 3 : Le Modèle Logique des Données (MLD)	3
1.5	5. Étape 4 : Le Langage de Définition de Données (SQL - LDD)	3
1.6	6. Étape 5 : Le Langage de Manipulation de Données (SQL - LMD)	5
1.7	7. Étape 6 : Développement d'Applications (Python & Lazarus)	6
2	Fiche 2 : Le Recueil de Questions-Réponses (40 Flashcards)	9
3	Fiche 3 : Quiz d'Évaluation & Fiches Mémo	16
3.1	Quiz d'Évaluation : 10 Questions types du Baccalauréat	16
3.2	3.2 Mémento de Synthèse Universelle	19
3.3	3.3 Les 5 Règles d'Or de l'Ingénierie de Bases de Données	19

1 Fiche 1 : Le Cours Complet — Conception & Implémentation

1.1 1. Présentation de l'Étude de Cas : Gestion de Médiathèque Scolaire

Pour ce chapitre de synthèse, nous étudions un cas complet et réaliste, emblématique des épreuves du Baccalauréat tunisien : **La Gestion d'une Médiathèque Scolaire d'Excellence**.

Cahier des charges & Règles de gestion : Une médiathèque scolaire souhaite informatiser la gestion de son patrimoine littéraire et numérique ainsi que le suivi des emprunts de ses abonnés. L'analyse des besoins a révélé les règles de gestion suivantes :

- **Abonnés :** Chaque adhérent possède un numéro d'adhérent unique, son nom, son prénom, son email, sa date d'adhésion et sa catégorie d'âge (Élève, Enseignant). Un adhérent peut être inscrit à un seul type d'abonnement.
- **Livres & Médias :** Chaque ouvrage possède un code livre unique, son titre, son auteur, son année d'édition et la quantité d'exemplaires disponibles.
- **Catégories :** Chaque livre appartient à une seule catégorie littéraire (ex : Roman, Science-Fiction, Informatique, Poésie), caractérisée par un code catégorie et un libellé. Une catégorie peut regrouper plusieurs livres.
- **Emprunts :** Un adhérent peut réaliser plusieurs emprunts de livres au fil du temps. Un livre peut être emprunté par plusieurs adhérents. Chaque emprunt est identifié par la date d'emprunt et enregistre également la date de retour effective, ainsi que l'état du livre à la restitution (Très Bon, Moyen, Dégradé). On doit pouvoir suivre chaque prêt individuellement.

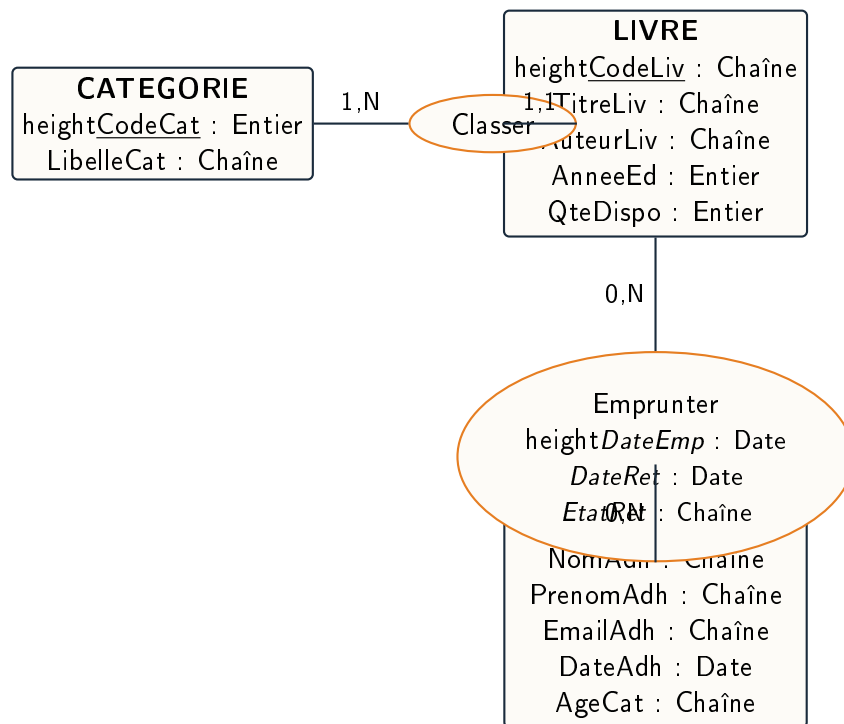
1.2 2. Étape 1 : Le Dictionnaire de Données (DD)

Le dictionnaire de données récapitule toutes les informations brutes à stocker, en leur associant des identifiants rigoureux, des types et leurs rôles précis dans la base.

Code Attribut	Désignation	Type Standard	Rôle / Contrainte
NumAdh	Numéro d'adhérent	Entier	Clé Primaire de l'adhérent (auto-incrément)
NomAdh	Nom de l'adhérent	Chaîne (50)	Obligatoire (NOT NULL)
PrenomAdh	Prénom de l'adhérent	Chaîne (50)	Obligatoire (NOT NULL)
EmailAdh	Adresse électronique	Chaîne (100)	Unique (UNIQUE)
DateAdh	Date d'inscription	Date	Valeur par défaut : date système (DEFAULT)
AgeCat	Catégorie d'âge / profil	Chaîne (15)	'Eleve' ou 'Enseignant' (CHECK)
CodeLiv	Code unique de l'ouvrage	Chaîne (10)	Clé Primaire de la table LIVRE
TitreLiv	Titre de l'ouvrage	Chaîne (150)	Obligatoire
AuteurLiv	Auteur principal	Chaîne (100)	Obligatoire
AnneeEd	Année de publication	Entier	Supérieure à 1800 (CHECK)
QteDispo	Quantité disponible	Entier	Supérieure ou égale à 0 (CHECK)
CodeCat	Code de la catégorie	Entier	Clé Primaire de la table CATEGORIE
LibelleCat	Libellé de la catégorie	Chaîne (50)	Unique et Obligatoire
DateEmp	Date d'emprunt	Date	Partie de la clé de l'emprunt
DateRet	Date de retour effective	Date	Peut être NULL tant que le livre n'est pas rendu
EtatRet	État à la restitution	Chaîne (15)	'Excellent', 'Bon', 'Moyen', 'Degrade'

1.3 3. Étape 2 : Le Modèle Conceptuel des Données (MCD)

Le MCD traduit graphiquement les concepts sémantiques. Les entités regroupent des propriétés, et les associations expriment les liaisons verbeuses munies de cardinalités de type (minimum, maximum).



1.4 4. Étape 3 : Le Modèle Logique des Données (MLD)

En appliquant les règles de dérivation Merise vers le modèle relationnel :

- Règle 1:N (Plusieurs-à-Un)** : L'association *Classer* relie une catégorie (1,N) à un livre (1,1). La clé primaire de la table parente CATEGORIE migre en tant que clé étrangère dans la table fille LIVRE.
- Règle N:N (Plusieurs-à-Plusieurs)** : L'association *Emprunter* comporte des cardinalités maximales à N des deux côtés. Elle donne naissance à une table d'association nommée EMPRUNT. Sa clé primaire est composée des clés primaires des tables associées (NumAdh et CodeLiv) complétée par la DateEmp afin de gérer les emprunts successifs d'un même adhérent pour un même livre.

Schéma Relationnel Textuel d'Excellence :

- **CATEGORIE** (CodeCat, LibelleCat)
- **LIVRE** (CodeLiv, TitreLiv, AuteurLiv, AnneeEd, QteDispo, CodeCat#)
- **ADHERENT** (NumAdh, NomAdh, PrenomAdh, EmailAdh, DateAdh, AgeCat)
- **EMPRUNT** (NumAdh#, CodeLiv#, DateEmp, DateRet, EtatRet)

1.5 5. Étape 4 : Le Langage de Définition de Données (SQL - LDD)

Voici le script SQL complet permettant d'instancier la base physique. Les tables sont ordonnées pour respecter les dépendances d'intégrité référentielle (les tables indépendantes d'abord).

Script SQL LDD d'Excellence

```
-- 1. Création de la table CATEGORIE
CREATE TABLE CATEGORIE (
    CodeCat INT PRIMARY KEY,
    LibelleCat VARCHAR(50) NOT NULL UNIQUE
);

-- 2. Création de la table LIVRE (avec Clé Étrangère référençant CATEGORIE)
CREATE TABLE LIVRE (
    CodeLiv VARCHAR(10) PRIMARY KEY,
    TitreLiv VARCHAR(150) NOT NULL,
    AuteurLiv VARCHAR(100) NOT NULL,
    AnneeEd INT CHECK (AnneeEd >= 1800),
    QteDispo INT CHECK (QteDispo >= 0),
    CodeCat INT NOT NULL,
    FOREIGN KEY (CodeCat) REFERENCES CATEGORIE(CodeCat)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

-- 3. Création de la table ADHERENT (avec contrainte de domaine CHECK)
CREATE TABLE ADHERENT (
    NumAdh INT PRIMARY KEY,
    NomAdh VARCHAR(50) NOT NULL,
    PrenomAdh VARCHAR(50) NOT NULL,
    EmailAdh VARCHAR(100) UNIQUE,
    DateAdh DATE DEFAULT CURRENT_DATE,
    AgeCat VARCHAR(15) NOT NULL CHECK (AgeCat IN ('Eleve', 'Enseignant'))
);

-- 4. Création de la table d'association EMPRUNT (avec clé primaire composée)
CREATE TABLE EMPRUNT (
    NumAdh INT,
    CodeLiv VARCHAR(10),
    DateEmp DATE,
    DateRet DATE,
    EtatRet VARCHAR(15) CHECK (EtatRet IN ('Excellent', 'Bon', 'Moyen', 'Degrade')),
    PRIMARY KEY (NumAdh, CodeLiv, DateEmp),
    FOREIGN KEY (NumAdh) REFERENCES ADHERENT(NumAdh)
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (CodeLiv) REFERENCES LIVRE(CodeLiv)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

1.6 6. Étape 5 : Le Langage de Manipulation de Données (SQL - LMD)

Afin de valider la maîtrise de l'interrogation, voici 10 requêtes types d'examen résolues de manière progressive et commentées :

Requête 1 : Liste simple *Sélectionner tous les livres édités après l'année 2020.*

```
SELECT CodeLiv, TitreLiv, AuteurLiv, AnneeEd
FROM LIVRE
WHERE AnneeEd > 2020
ORDER BY TitreLiv ASC;
```

Requête 2 : Recherche de motif avec LIKE *Afficher les adhérents dont le nom commence par la lettre 'B'.*

```
SELECT NumAdh, NomAdh, PrenomAdh
FROM ADHERENT
WHERE NomAdh LIKE 'B%';
```

Requête 3 : Jointure classique *Lister les titres des livres et les libellés de catégories correspondants.*

```
SELECT L.TitreLiv, C.LibelleCat
FROM LIVRE L, CATEGORIE C
WHERE L.CodeCat = C.CodeCat;
```

Requête 4 : Groupement d'enregistrements (GROUP BY) *Compter le nombre total de livres par catégorie littéraire.*

```
SELECT C.LibelleCat, COUNT(L.CodeLiv) AS NombreLivres
FROM CATEGORIE C, LIVRE L
WHERE C.CodeCat = L.CodeCat
GROUP BY C.LibelleCat;
```

Requête 5 : Regroupement avec filtrage sur agrégat (HAVING) *Sélectionner les libellés des catégories qui possèdent strictement plus de 5 livres enregistrés.*

```
SELECT C.LibelleCat, COUNT(L.CodeLiv) AS Total
FROM CATEGORIE C, LIVRE L
WHERE C.CodeCat = L.CodeCat
GROUP BY C.LibelleCat
HAVING COUNT(L.CodeLiv) > 5;
```

Requête 6 : Requête imbriquée (Sous-requête) *Lister les adhérents qui n'ont jamais emprunté le moindre livre.*

```
SELECT NumAdh, NomAdh, PrenomAdh
FROM ADHERENT
WHERE NumAdh NOT IN (SELECT DISTINCT NumAdh FROM EMPRUNT);
```

Requête 7 : Multi-jointure complexe *Afficher la liste des élèves (Nom, Prénom) ayant actuellement un livre emprunté non rendu (DateRet est NULL), avec le titre du livre.*

```
SELECT A.NomAdh, A.PrenomAdh, L.TitreLiv, E.DateEmp
FROM ADHERENT A, LIVRE L, EMPRUNT E
WHERE A.NumAdh = E.NumAdh
      AND L.CodeLiv = E.CodeLiv
      AND E.DateRet IS NULL
      AND A.AgeCat = 'Eleve';
```

Requête 8 : Mise à jour conditionnelle *Mettre à jour la date de retour effective de l'emprunt de l'adhérent 105 pour le livre 'LIV50' au 5 septembre 2026, avec un état 'Bon'.*

```
UPDATE EMPRUNT
SET DateRet = '2026-09-05', EtatRet = 'Bon'
WHERE NumAdh = 105
      AND CodeLiv = 'LIV50'
      AND DateRet IS NULL;
```

Requête 9 : Suppression sous contrainte *Supprimer les adhérents de catégorie d'âge 'Eleve' dont l'inscription remonte à plus de 5 ans.*

```
DELETE FROM ADHERENT
WHERE AgeCat = 'Eleve'
      AND DateAdh < '2021-09-05';
```

Requête 10 : Calculs cumulatifs avancés *Trouver le titre du livre le plus emprunté de toute la médiathèque.*

```
SELECT L.TitreLiv, COUNT(E.NumAdh) AS TotalPrets
FROM LIVRE L, EMPRUNT E
WHERE L.CodeLiv = E.CodeLiv
GROUP BY L.TitreLiv
HAVING COUNT(E.NumAdh) >= ALL (
    SELECT COUNT(NumAdh)
    FROM EMPRUNT
    GROUP BY CodeLiv
);
```

1.7 7. Étape 6 : Développement d'Applications (Python & Lazarus)

La programmation d'interfaces logicielles connectées à la base se décline sous les deux langages du Baccalauréat tunisien.

A. Implémentation en Python d'Excellence (sqlite3) : Ce script simule la fonction métier sécurisée de création d'un nouvel emprunt, en intégrant une **requête paramétrée** pour éradiquer les risques d'Injections SQL, et une transaction atomique.

Script Python de Liaison Applicative

```
import sqlite3
from datetime import date

def enregistrer_emprunt(num_adh, code_liv):
```

```
conn = None
try:
    # Établir la connexion physique avec le fichier de base de données
    conn = sqlite3.connect('/workspace/scratch/mediatheque.db')
    cur = conn.cursor()

    # 1. Vérification de la disponibilité du livre
    query_check = "SELECT QteDispo FROM LIVRE WHERE CodeLiv = ?"
    cur.execute(query_check, (code_liv,))
    row = cur.fetchone()

    if row is None:
        print("Erreur : Livre inexistant.")
        return False

    qte = row[0]
    if qte <= 0:
        print("Erreur : Ce livre n'est plus disponible en stock.")
        return False

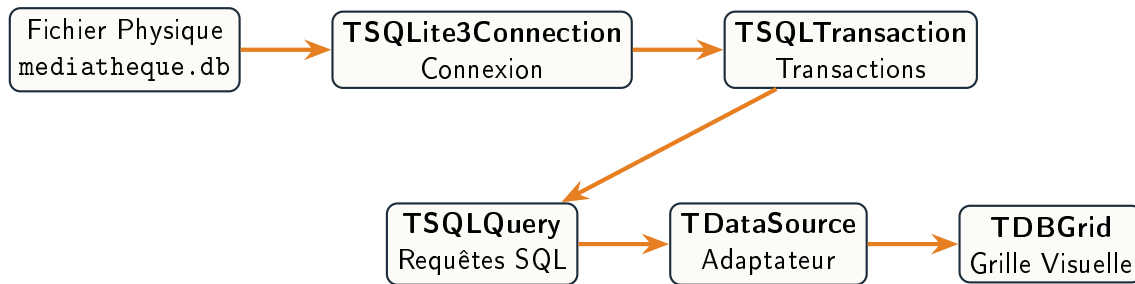
    # 2. Insertion de la ligne d'emprunt (Requête Paramétrée Sécurisée)
    query_insert = "INSERT INTO EMPRUNT (NumAdh, CodeLiv, DateEmp, DateRet, EtatRet) VALUES (?, ?, ?, ?, ?)"
    cur.execute(query_insert, (num_adh, code_liv, date.today().isoformat(), 1, 0))

    # 3. Décrémentation du stock disponible
    query_update = "UPDATE LIVRE SET QteDispo = QteDispo - 1 WHERE CodeLiv = ?"
    cur.execute(query_update, (code_liv,))

    # Validation de la transaction globale (Atomicité)
    conn.commit()
    print("Emprunt enregistré avec succès !")
    return True

except sqlite3.Error as e:
    if conn:
        conn.rollback() # Annulation en cas de bug (Rollback)
        print(f"Erreur système durant la transaction : {e}")
        return False
finally:
    if conn:
        conn.close()
```

B. Interfaçage Visuel en Lazarus (Composants SQLdb) : Le développement sous Lazarus repose sur la mise en place d'une chaîne logique de composants non visuels connectés à la grille graphique de l'utilisateur.



Propriétés clés des composants :

- `TSQLite3Connection1.DatabaseName ← '/workspace/scratch/mediatheque.db'`
- `TSQLite3Connection1.Transaction ← TSQLTransaction1`
- `TSQLQuery1.Database ← TSQLite3Connection1`
- `TSQLQuery1.Transaction ← TSQLTransaction1`
- `TDataSource1.DataSet ← TSQLQuery1`
- `TDBGrid1.DataSource ← TDataSource1`

Code Pascal de modification structurelle dynamique : Ce fragment de code permet de charger dynamiquement une requête de sélection et d'ouvrir les résultats sur la grille visuelle TDBGrid1 lors d'un clic sur un bouton :

Code Pascal Lazarus

```

procedure TForm1.BtnAfficherClick(Sender: TObject);
begin
  SQLQuery1.Close; // Libération de la mémoire de l'ancienne requête
  SQLQuery1.SQL.Clear; // Nettoyage du texte SQL

  // Réécriture de la requête dynamique
  SQLQuery1.SQL.Add('SELECT L.TitreLiv, C.LibelleCat, L.QteDispo ');
  SQLQuery1.SQL.Add('FROM LIVRE L, CATEGORIE C ');
  SQLQuery1.SQL.Add('WHERE L.CodeCat = C.CodeCat AND L.QteDispo > 0');

  SQLQuery1.Open; // Activation de la requête de sélection (Active := True)
end;
  
```

2 Fiche 2 : Le Recueil de Questions-Réponses (40 Flashcards)

Ce recueil comprend 40 Flashcards d'apprentissage actif (Active Recall) conçues spécifiquement autour de l'Étude de Cas globale de la médiathèque pour réviser et acquérir la note maximale au Baccalauréat.

Flashcard 1

Question : Quel est le but premier de la modélisation conceptuelle (MCD) lors d'une étude de cas de base de données ?

Réponse : Représenter l'aspect sémantique des données de l'organisation sans contraintes techniques ni matérielles, en modélisant fidèlement les concepts métiers réels sous forme d'Entités et d'Associations.

Flashcard 2

Question : Qu'est-ce qu'une occurrence d'entité et donnez un exemple basé sur la table ADHERENT ?

Réponse : C'est un exemplaire unique, un enregistrement physique d'une entité. Exemple : L'adhérent ayant pour numéro d'adhérent 102, se nommant 'Sami', et de catégorie 'Eleve'.

Flashcard 3

Question : Pourquoi la clé primaire de la table CATEGORIE ne migre-t-elle pas dans ADHERENT ?

Réponse : Parce qu'il n'existe aucune association directe ou contrainte logique entre un adhérent et une catégorie littéraire au niveau du MCD de notre médiathèque.

Flashcard 4

Question : Quelle est la différence entre une cardinalité 1,1 et une cardinalité 1,N dans notre étude de cas ?

Réponse : La cardinalité 1,1 (entre LIVRE et Classer) signifie qu'un livre appartient obligatoirement à une seule et unique catégorie. La cardinalité 1,N (entre CATEGORIE et Classer) signifie qu'une catégorie littéraire regroupe un ou plusieurs livres.

Flashcard 5

Question : Qu'est-ce qu'une contrainte d'intégrité de domaine et comment l'implémente-t-on en SQL pour l'âge ?

Réponse : C'est une règle de validation qui restreint les valeurs autorisées d'une colonne. On l'implémente via la clause CHECK (Exemple : CHECK (AgeCat IN ('Eleve', 'Enseignant'))).

Flashcard 6

Question : Pourquoi a-t-on besoin d'une clé primaire composée pour la table EMPRUNT ?

Réponse : Car un même adhérent peut emprunter différents livres, et un même livre peut être emprunté par différents adhérents. La combinaison de NumAdh, CodeLiv et DateEmp garantit l'unicité stricte de chaque emprunt.

Flashcard 7

Question : Quelle est la conséquence d'oublier la clause `ON DELETE CASCADE` sur la table `EMPRUNT` ?

Réponse : Si l'on tente de supprimer un adhérent qui possède des emprunts actifs dans la base, le SGBD refusera catégoriquement la suppression pour éviter de créer des lignes orphelines, ce qui violerait l'intégrité référentielle.

Flashcard 8

Question : Que fait précisément l'option comportementale `ON UPDATE CASCADE` sur une clé étrangère ?

Réponse : Elle permet de propager automatiquement toute modification de la valeur de la clé primaire (dans la table parente) vers la clé étrangère correspondante dans les tables filles liées.

Flashcard 9

Question : Pourquoi choisit-on le type `VARCHAR(10)` pour `CodeLiv` au lieu d'un type `INT` ?

Réponse : Parce que les codes de livres comportent souvent des caractères alphanumériques d'identification (par exemple `'LIV204'`), ce qui nécessite une chaîne de caractères de taille variable plutôt qu'un nombre entier.

Flashcard 10

Question : Quelle est la différence entre les clauses `WHERE` et `HAVING` en SQL LMD ?

Réponse : `WHERE` filtre individuellement les lignes brutes d'une table avant tout regroupement. `HAVING` filtre globalement les groupes formés par la clause `GROUP BY` en se basant sur des fonctions d'agrégation.

Flashcard 11

Question : Pourquoi est-il obligatoire de déclarer la table CATEGORIE avant la table LIVRE dans le script LDD ?

Réponse : Parce que la table LIVRE contains une clé étrangère qui fait référence à la clé primaire de la table CATEGORIE. La table cible doit obligatoirement exister au préalable pour que le SGBD puisse créer le lien d'intégrité.

Flashcard 12

Question : Comment peut-on tester l'existence de valeurs non renseignées dans une colonne SQL ?

Réponse : En utilisant l'opérateur IS NULL (Exemple : WHERE DateRet IS NULL). L'opérateur d'égalité classique = ne fonctionne jamais avec la valeur spéciale NULL.

Flashcard 13

Question : Quel est l'intérêt de la clause UNIQUE appliquée à l'attribut EmailAdh ?

Réponse : Elle interdit à deux adhérents de posséder la même adresse email dans la base de données, assurant l'unicité logique de l'identification électronique.

Flashcard 14

Question : Qu'est-ce que l'atomicité d'un attribut dans une base de données relationnelle (1ère Forme Normale) ?

Réponse : C'est le principe selon lequel chaque colonne d'une table doit contenir une valeur unique indivisible (Exemple : interdiction de stocker plusieurs numéros de téléphone dans un même champ).

Flashcard 15

Question : Quelle requête SQL permet d'ajouter un nouvel adhérent dans la base de données ?

Réponse : INSERT INTO ADHERENT VALUES (101, 'Khemiri', 'Ahmed', 'ahmed@email.com', '2026-09-05', 'Eleve');

Flashcard 16

Question : Comment supprime-t-on définitivement la table EMPRUNT de la base de données ?

Réponse : En exécutant l'instruction LDD DROP TABLE EMPRUNT;. Cela détruira à la fois sa structure et la totalité de son contenu de manière irréversible.

Flashcard 17

Question : Pourquoi la requête SELECT * FROM LIVRE, CATEGORIE; est-elle considérée comme une erreur majeure d'interrogation ?

Réponse : Parce qu'elle n'indique aucune condition de jointure dans la clause WHERE, générant ainsi un produit cartésien associant inutilement chaque livre à chaque catégorie de la base.

Flashcard 18

Question : À quoi sert l'opérateur AS dans une requête de sélection SQL ?

Réponse : Il définit un alias, c'est-à-dire un nom temporaire plus explicite attribué à une colonne calculée ou à une table pour simplifier l'affichage ou l'écriture du code de la requête.

Flashcard 19

Question : Quelle est la syntaxe SQL pour modifier le type de l'attribut TitreLiv de la table LIVRE en VARCHAR(200) ?

Réponse : ALTER TABLE LIVRE MODIFY TitreLiv VARCHAR(200) NOT NULL; (ou ALTER COLUMN selon le SGBD).

Flashcard 20

Question : Que se passe-t-il si un adhérent tente d'emprunter un livre dont la QteDispo est de 0 ?

Réponse : Notre script Python bloque l'opération en affichant un message d'erreur et refuse d'exécuter l'insertion afin de maintenir la cohérence physique des stocks réels de la médiathèque.

Flashcard 21

Question : Pourquoi les requêtes paramétrées bloquent-elles définitivement les injections SQL en Python ?

Réponse : Parce qu'elles séparent l'interprétation logique de la structure de la requête SQL des données d'entrée fournies par l'utilisateur, traitant ces dernières uniquement comme des constantes inoffensives.

Flashcard 22

Question : Quelle est la différence entre la méthode commit() et la méthode rollback() en programmation Python ?

Réponse : commit() applique définitivement les modifications d'écriture en base physique, tandis que rollback() annule instantanément toutes les modifications en cours depuis le dernier commit en cas de plantage.

Flashcard 23

Question : Quel composant Lazarus est responsable d'exécuter la connexion physique à la base SQLite3 ?

Réponse : Le composant TSQLite3Connection, dont la propriété DatabaseName doit être initialisée avec le chemin d'accès absolu du fichier de base de données physique.

Flashcard 24

Question : Quel est le rôle du composant TDataSource dans Lazarus ?

Réponse : Faire office de pont logique reliant le composant d'accès aux données (TSQLQuery) aux composants d'affichage graphique de l'interface utilisateur (TDBGrid).

Flashcard 25

Question : Pourquoi appelle-t-on SQLQuery1.Close avant de redéfinir son texte SQL dans Lazarus ?

Réponse : Pour vider les résultats de la requête précédente en mémoire vive et réinitialiser correctement le composant afin de préparer la compilation et l'exécution du nouveau script SQL.

Flashcard 26

Question : Dans Lazarus, quelle est la différence entre l'appel à la méthode Open et la méthode ExecSQL d'un TSQLQuery ?

Réponse : Open (ou Active := True) est requise exclusivement pour les requêtes d'interrogation de type SELECT (qui retournent un jeu de résultats). ExecSQL est requise pour les requêtes d'écriture de type LMD (INSERT, UPDATE, DELETE).

Flashcard 27

Question : Comment déclare-t-on un paramètre dynamique dans une requête SQL écrite sous Lazarus ?

Réponse : En préfixant l'identifiant du paramètre dynamique par le symbole double-point (:) dans le corps textuel de la requête SQL (Exemple : WHERE CodeLiv = :MonCode).

Flashcard 28

Question : Quelle propriété d'un composant TSQLQuery permet d'attribuer une valeur à un paramètre nommé sous Lazarus ?

Réponse : La propriété Params.ParamByName('NomParam').AsDataType (Exemple : SQLQuery1.Params.ParamByName('MonCode').AsString := 'LIV20';).

Flashcard 29

Question : Que se passe-t-il sous Lazarus si l'on oublie d'activer le composant TSQLTransaction ?

Réponse : Aucune donnée de sélection ne sera affichée dans la grille visuelle TDBGrid, et toute tentative de requête d'écriture déclenchera une exception fatale indiquant l'absence d'une transaction active.

Flashcard 30

Question : Comment peut-on forcer la validation des modifications en base de données physique depuis Lazarus ?

Réponse : En appelant la méthode Commit du composant TSQLTransaction (Exemple : SQLTransaction1.Commit;).

Flashcard 31

Question : Comment l'intégrité de domaine est-elle violée si l'on tente d'insérer l'adhérent 'Ami' avec la valeur 'Senior' pour AgeCat ?

Réponse : La contrainte de validation CHECK (AgeCat IN ('Eleve', 'Enseignant')) de la table rejette l'insertion car 'Senior' ne fait pas partie des valeurs explicitement autorisées pour cette colonne.

Flashcard 32

Question : Pourquoi la jointure externe est-elle utile pour notre étude de cas ?

Réponse : Elle permet de lister toutes les catégories de livres de la base de données, y compris les catégories littéraires nouvellement créées qui ne contiennent encore aucun livre associé.

Flashcard 33

Question : À quoi sert la directive système {\$I-} et {\$I+} dans les manipulations Lazarus ?

Réponse : Elle désactive puis réactive le contrôle automatique des erreurs d'Entrées/Sorties, permettant au programme de gérer manuellement les plantages matériels ou réseaux via la variable IOResult.

Flashcard 34

Question : Quelle commande SQL permet de supprimer l'adhérent 102 sans impacter le reste du dictionnaire ?

Réponse : DELETE FROM ADHERENT WHERE NumAdh = 102;. Le SGBD supprimera également ses emprunts associés en cascade si ON DELETE CASCADE est configuré.

Flashcard 35

Question : Qu'est-ce qu'une table de dictionnaire de données système et que contient-elle ?

Réponse : C'est une métatable interne gérée par le SGBD qui stocke la structure de la base : les définitions des tables, les types de colonnes, et les contraintes d'intégrité déclarées.

Flashcard 36

Question : Pourquoi le mot-clé DISTINCT est-il indispensable pour compter le nombre d'adhérents actifs ?

Réponse : Car un même adhérent peut réaliser de multiples emprunts. Écrire `COUNT(NumAdh)` dans la table `EMPRUNT` compterait les prêts, tandis que `COUNT(DISTINCT NumAdh)` compte les adhérents uniques.

Flashcard 37

Question : Quelle est la complexité temporelle de la sélection d'un livre via sa clé primaire en base de données ?

Réponse : Elle s'exécute en temps constant ($O(1)$) car les clés primaires font automatiquement l'objet d'une indexation physique par le SGBD (structure en arbre indexé).

Flashcard 38

Question : Comment peut-on restaurer la base de données après un crash de disque dur ?

Réponse : En chargeant le dernier fichier de sauvegarde logique (*Dump*) et en appliquant les opérations validées consignées dans le journal des transactions jusqu'à l'instant de la panne.

Flashcard 39

Question : Quel est l'impact de l'absence de l'option `ON UPDATE CASCADE` lors du renommage d'un code de livre ?

Réponse : Toute tentative de mise à jour d'un identifiant de livre sera catégoriquement rejetée par le SGBD car des lignes d'emprunt existantes font référence à cet ancien identifiant.

Flashcard 40

Question : Quelle propriété ACID d'une transaction garantit que les modifications partielles d'un emprunt ne polluent pas la base ?

Réponse : L'**Atomicité**. Elle garantit que soit l'ensemble des requêtes de la transaction s'exécutent avec succès (insertion et mise à jour de stock), soit aucune n'est enregistrée.

3 Fiche 3 : Quiz d'Évaluation & Fiches Mémo

3.1 Quiz d'Évaluation : 10 Questions types du Baccalauréat

Question 1 — QCM

Quelle est la clé primaire de la table d'association EMPRUNT déduite du MCD de l'étude de cas ?

- A) NumAdh
- B) NumAdh, CodeLiv
- C) NumAdh, CodeLiv, DateEmp

Correction & Justification :

La bonne réponse est **C**. L'adhérent peut emprunter le même livre à différentes dates au fil de l'année. Si la clé était uniquement (NumAdh, CodeLiv), il serait techniquement impossible de prêter une deuxième fois le même ouvrage à cet adhérent une fois le premier exemplaire restitué.

Question 2 — Vrai ou Faux

La contrainte d'intégrité référentielle interdit d'insérer un enregistrement dans la table LIVRE avec un CodeCat qui n'existe pas dans la table CATEGORIE.

Correction & Justification :

VRAI. C'est le principe fondamental de la clé étrangère. L'attribut CodeCat de la table LIVRE fait référence à la clé primaire de la table CATEGORIE. Chaque valeur saisie dans CodeCat de LIVRE doit obligatoirement exister au préalable dans la table cible.

Question 3 — QCM

Quelle fonction d'agrégation SQL permet de déterminer le nombre total d'emprunts en cours ?

- A) SUM(DateEmp)
- B) COUNT(NumAdh)
- C) AVG(CodeLiv)

Correction & Justification :

La bonne réponse est **B**. La fonction d'agrégation COUNT calcule le nombre de lignes correspondant au prédicat. SUM et AVG ne s'appliquent qu'à des valeurs numériques.

Question 4 — Vrai ou Faux

Dans Lazarus, le composant TDataSource doit pointer vers le composant de connexion pour pouvoir afficher la grille.

Correction & Justification :

FAUX. Le composant TDataSource fait office d'adaptateur logique et doit pointer vers le composant d'accès aux données TSQLQuery via sa propriété DataSet. C'est la grille TDBGrid qui pointe vers le TDataSource.

Question 5 — QCM

Quelle instruction SQL permet d'ajouter la contrainte de domaine 'Eleve' ou 'Enseignant' à la colonne AgeCat ?

- A) ALTER TABLE ADHERENT ADD CHECK (AgeCat IN ('Eleve', 'Enseignant'));
- B) ALTER TABLE ADHERENT ADD CONSTRAINT AgeCat IN ('Eleve', 'Enseignant');
- C) CREATE CHECK ON ADHERENT (AgeCat IN ('Eleve', 'Enseignant'));

Correction & Justification :

La bonne réponse est **A**. L'instruction LDD SQL officielle pour ajouter une contrainte de domaine sur une table existante est `ALTER TABLE` suivie de la clause `ADD CHECK` définissant le prédicat.

Question 6 — Vrai ou Faux

Si l'on exécute une suppression d'une catégorie via la commande `DELETE FROM CATEGORIE WHERE CodeCat = 3;`, et que des livres y font référence, la base de données décrètera une erreur système s'il n'y a pas d'option `CASCADE`.

Correction & Justification :

VRAI. En l'absence de comportement `ON DELETE CASCADE` (ou si `ON DELETE RESTRICT` est spécifié), le SGBD bloque la suppression pour interdire la création de livres "orphelins" (ne possédant plus de catégorie valide).

Question 7 — QCM

Quelle clause SQL est indispensable pour filtrer les résultats sur une fonction d'agrégation ?

- A) `WHERE`
- B) `HAVING`
- C) `GROUP BY`

Correction & Justification :

La bonne réponse est **B**. La clause `HAVING` est spécialement conçue pour filtrer les groupes formés par la clause `GROUP BY` sur des critères d'agrégation (ex : `HAVING COUNT(*) > 5`). `WHERE` ne peut pas filtrer sur des agrégations.

Question 8 — Vrai ou Faux

La méthode `cur.fetchall()` d'un curseur Python renvoie la totalité des lignes sélectionnées sous la forme d'un unique dictionnaire.

Correction & Justification :

FAUX. La méthode `fetchall()` renvoie la totalité des lignes sous la forme d'une **liste de tuples** Python (chaque tuple représentant une ligne d'enregistrement de la table).

Question 9 — QCM

Quelle commande SQL permet de vider l'ensemble des enregistrements de la table d'association `EMPRUNT` tout en conservant intacte sa structure ?

- A) `DROP TABLE EMPRUNT;`
- B) `DELETE FROM EMPRUNT;`
- C) `REMOVE * FROM EMPRUNT;`

Correction & Justification :

La bonne réponse est **B**. `DELETE FROM` supprime le contenu logique d'une table sans altérer sa définition dans le dictionnaire de données. `DROP TABLE` détruit la structure même.

Question 10 — Vrai ou Faux

Toute modification apportée à la base de données à l'aide d'un script Python connecté via `sqlite3` est immédiatement et définitivement écrite en base physique, même en l'absence de l'instruction `commit()`.

Correction & Justification :

FAUX. En Python, le moteur transactionnel est activé par défaut. Les modifications d'écriture (`INSERT`, `UPDATE`, `DELETE`) restent stockées dans un tampon mémoire temporaire et sont perdues si l'on ferme la connexion sans appeler explicitement la méthode `commit()`.

3.2 3.2 Mémento de Synthèse Universelle

Ce tableau synoptique offre un aperçu d'ingénierie complet des étapes de l'Étude de Cas, indispensables pour la mémorisation absolue.

Étape d'Ingénierie	Livrable Produit	Outil & Rôle Système
1. Spécification	Cahier des charges / Dictionnaire de données	Définition textuelle des propriétés, types de données et règles métiers de validation.
2. Modélisation	Modèle Conceptuel des Données (MCD)	Analyse sémantique graphique : isolation des Entités, Associations et des Cardinalités.
3. Structuration	Modèle Logique des Données (MLD)	Traduction relationnelle textuelle : génération des Clés Primaires (PK) et Étrangères (FK).
4. Implémentation	Script SQL LDD (CREATE TABLE)	Création des tables physiques du SGBD avec déclaration des contraintes d'intégrité.
5. Exploitation	Requêtes SQL LMD (SELECT, INSERT)	Sélection, filtrage, regroupement et modification dynamique du contenu de la base.
6. Interfaçage	Code de liaison applicatif (Python/Lazarus)	Connexions physiques, exécution de requêtes paramétrées sécurisées et affichage visuel.

3.3 3.3 Les 5 Règles d'Or de l'Ingénierie de Bases de Données

Pour décrocher la note maximale (20/20) le jour du Baccalauréat tunisien, appliquez systématiquement ces principes d'excellence :

- Vérifiez l'ordre de création LDD** : Créez toujours les tables indépendantes (qui ne contiennent aucune clé étrangère, comme CATEGORIE et ADHERENT) en premier. Les tables filles dépendent de l'existence de leurs parents.
- Ne faites jamais de produit cartésien** : Pour chaque jointure de tables dans la clause FROM, ajoutez obligatoirement la condition de clé correspondante (TableA.PK = TableB.FK) dans le WHERE pour éviter d'associer inutilement toutes les lignes.
- Sécurisez les requêtes applicatives** : N'utilisez jamais la concaténation de chaînes de caractères pour injecter des valeurs utilisateur dans du code SQL. Utilisez uniquement des **requêtes paramétrées** (? en Python, : en Lazarus) pour éradiquer les risques d'Injections SQL.
- Utilisez le Commit transactionnel** : En programmation Python comme Lazarus, validez systématiquement vos transactions d'écriture à l'aide de la méthode `commit()` ou `Commit;`. Sans cela, les modifications ne seront jamais pérennisées.
- Respectez la clé primaire de relation N:N** : Lors de la conversion d'une association Plusieurs-à-Plusieurs (N:N), la table d'association générée possède une clé primaire composée d'au moins les clés primaires des deux tables parentes. Ne réduisez jamais cette clé à un seul attribut.